arXiv:2312.00246v2 [cs.LG] 17 Feb 2024

Directions of Curvature as an Explanation for Loss of Plasticity

Alex Lewandowski [1] **Haruto Tanaka** [1] **Dale Schuurmans** [1 2 3] **Marlos C. Machado** [1 3]

Abstract

Loss of plasticity is a phenomenon in which neural networks lose their ability to learn from new experience. Despite being empirically observed in several problem settings, little is understood about the mechanisms that lead to loss of plasticity. In this paper, we offer a consistent explanation for loss of plasticity: Neural networks lose directions of curvature during training and that loss of plasticity can be attributed to this reduction in curvature. To support such a claim, we provide a systematic investigation of loss of plasticity across continual learning tasks using MNIST, CIFAR-10 and ImageNet. Our findings illustrate that loss of curvature directions coincides with loss of plasticity, while also showing that previous explanations are insufficient to explain loss of plasticity in all settings. Lastly, we show that regularizers which mitigate loss of plasticity also preserve curvature, motivating a simple distributional regularizer that proves to be effective across the problem settings we considered.

1. Introduction

A longstanding goal of machine learning research is to develop algorithms that can learn *continually* and cope with unforeseen changes in the data distribution (Ring, 1994; Thrun, 1998). Current learning algorithms, however, struggle to learn from dynamically changing targets and are unable to adapt gracefully to unforeseen changes in the distribution during the learning process (Zilly et al., 2021; Abbas et al., 2023; Lyle et al., 2023; Dohare et al., 2023a). Such limitations can be seen to be a byproduct of assuming, one way or another, the problem is stationary. Recently, there has been growing recognition of the fact that there are limitations to what can be learned from a fixed and unchanging dataset (Hoffmann et al., 2022) and that there are implicit non-stationarities in many problems of interest (Igl et al., 2021).

[1]Department of Computing Science, University of Alberta, Edmonton, Canada [2]Google DeepMind [3]Canada CIFAR AI Chair. Correspondence to: Alex Lewandowski <lewandowski@ualberta.ca>.

The concept of plasticity has been receiving growing attention in the continual learning literature, where the loss of plasticity—a reduction in the ability to learn new things (Dohare et al., 2023a; Lyle et al., 2023)—has been noted as a critical shortcoming in current learning algorithms. That is, learning algorithms that are performant in the non-continual learning setting, and more specifically neural networks, often struggle when applied to continual learning problems, exhibiting a striking loss of plasticity such that learning slows down or even halts after successive changes in the learning distribution. Such a loss of plasticity can be readily observed in settings where a neural network must continue to learn after changes occur in the observations or targets.

Several aspects of a learning algorithm have been found to contribute to, or mitigate, loss of plasticity. Examples include the optimizer (Dohare et al., 2023a), the step-size (Ash & Adams, 2020; Berariu et al., 2021), the number of updates (Lyle et al., 2023), the activation function (Abbas et al., 2023), and the use of specific regularizers (Dohare et al., 2021; Kumar et al., 2023; Lyle et al., 2021). Such factors hint that there might be simpler underlying mechanisms mechanisms for loss of plasticity. For example, the success of several methods that regularize neural networks towards properties of the initialization suggests that some property of the initialization mitigates loss of plasticity. Unfortunately, no such property has yet been identified. Some factors that have been found to correlate with loss of plasticity include, a decrease in the gradient or update norm (Abbas et al., 2023), neuron dormancy (Sokar et al., 2023), and an increase in the norm of the parameters (Nikishin et al., 2022).

In this paper, we propose that loss of plasticity can be explained by a loss of curvature directions. Our work contributes to a growing literature on the importance of curvature for understanding neural network dynamics (Cohen et al., 2021; Hochreiter & Schmidhuber, 1997; Fort & Ganguli, 2019). Within the continual learning and plasticity literature, the assertion that curvature is related to plasticity is relatively new (Lyle et al., 2023). In contrast to the general assertion that curvature is related to plasticity, our work specifically posits that loss of curvature directions explains loss of plasticity. In particular, we provide empirical evidence that supports the claim that loss of plasticity co-occurs with a reduction in the rank of the Hessian of the training objective at the beginning of a new task.

More specifically, this work improves the understanding of loss of plasticity in continual supervised learning by:

1. Surveying previous explanations for loss of plasticity. We provide counterexamples showing that existing explanations are not consistent, that is, they do not explain loss of plasticity in all situations it occurs.
2. Proposing that loss of curvature directions, measured as the reduction in the rank of the Hessian of the training objective, is a consistent explanation for loss of plasticity. We demonstrate that loss of curvature directions coincides with loss of plasticity across all factors and benchmarks that we consider.
3. Introducing a Wasserstein regularizer that keeps the distribution of weights close to the initialization distribution. This Wasserstein regularizer allows the parameters to move further from initialization while preserving curvature for successive tasks. Learning with the Wasserstein regularizer requires fewer iterations and achieves a lower error compared to other regularizers.

2. Factors and Explanations for Loss of Plasticity

Before defining what we mean by loss of plasticity, we outline the continual supervised learning problem setting we study. We assume the learning algorithm operates in a minibatch setting, processing M observation-target pairs, $\{x_i, y_i\}_{i=1}^{M}$, and updating the neural network parameters, θ, after each minibatch. In continual supervised learning, there is a periodic and regular change every U updates to the distribution generating the observations or targets. For every U updates, the neural network must minimize an objective defined over a new distribution—we refer to this new distribution as a *task*. The problem setting is designed so that the task at any point in time has the same difficulty.[1] We are primarily interested in the error at the the end of task K averaged across all observations in that task, $J_K = J(\theta_{UK}) = \mathbb{E}_{p_K}\left[\ell(f_{\theta_{UK}}(x), y)\right]$, for some loss function ℓ, and task specific data distribution p_K.

Although loss of plasticity is an empirically observed phenomenon, the way it is measured in the literature can vary. In this paper, we use loss of plasticity to refer to the phenomenon that J_K increases rather than decreases as a function of K. Some works evaluate learning and plasticity with the average online error over the learning trajectory within a task (e.g., Elsayed & Mahmood, 2023; Dohare et al., 2023a; Kumar et al., 2023). While the two are related, we focus on the error at the end of the task to remove the effect of the unavoidable error increase at the beginning of a subsequent task. If we were to consider the large initial error, we might infer loss of plasticity in the average online error even if the error at the end of a task is constant (see Appendix C.1). Because the error at the end of a task increases as more tasks are seen, this means that the neural network is struggling to learn from the new experience given by the subsequent task.

[1] A suitably initialized neural network should be able to equally minimize the objective for any of the tasks we consider.

2.1. Factors That Can Contribute to Loss of Plasticity

Given a concrete notion of plasticity, we reiterate that the underlying mechanisms leading to loss of plasticity have been so-far elusive. This is partly because multiple factors can potentially contribute to, or mitigate, loss of plasticity. In this section, we summarize some of these potential factors before surveying previous explanations for the underlying mechanism behind loss of plasticity.

Optimizer Optimizers that were designed and tuned for stationary distributions can exacerbate loss of plasticity in non-stationary settings. For instance, the work by Lyle et al. (2023) showed empirically that Adam (Kingma & Ba, 2015) can be unstable on a subsequent task due to its momentum and scaling from a previous task.

Step-size In addition to the optimizer, the step-size is a crucial factor in both contributing to and mitigating loss of plasticity. The study by Berariu et al. (2021), for example, suggests that loss of plasticity is preventable by amplifying the randomness of gradients with a larger step-size. These findings extend to other hyper-parameters of the optimizer. Properly tuned hyper-parameters for Adam, for example, can mitigate loss of plasticity which leads to policy collapse in reinforcement learning (Dohare et al., 2023b; Lyle et al., 2023).

Update budget Continual supervised learning experiments, including those below, use a fixed number of update steps per task (e.g., Abbas et al., 2023; Elsayed & Mahmood, 2023; Javed & White, 2019). Despite the fact that the individual tasks themselves are of the same difficulty, the neural network might not be able to escape its task-specific initialization within the pre-determined update budget. Lyle et al. (2023) show that, as the number of update steps increase in a first task, learning slows down on a subsequent task, requiring even more update steps on the subsequent task to reach the same training error.

Activation function One major factor that can contribute or mitigate loss of plasticity is the activation function. Work by Abbas et al. (2023) suggests that, in the reinforcement learning setting, loss of plasticity occurs because of an increasing portion of hidden units being set to zero by `ReLU` activations (Fukushima, 1975; Nair & Hinton, 2010). The authors then show that `CReLU` (Shang et al., 2016) prevents saturation, mitigating loss of plasticity almost entirely. However, other works have shown that loss of plasticity can still occur with non-saturating activation functions (Dohare et al.,

2021; 2023a) such as `leaky-ReLU` (Xu et al., 2015).

Properties of the objective function and the regularizer The objective function being optimized greatly influences the optimization landscape and, hence, plasticity (Lyle et al., 2021; 2023; Ziyin, 2023). Regularization is one modification to the objective function that helps mitigate loss of plasticity. For example, when weight decay is properly tuned, it can help mitigate loss of plasticity (Dohare et al., 2023a). Another regularizer that mitigates loss of plasticity is regenerative regularization, which regularizes towards the parameter initialization (Kumar et al., 2023).

2.2. Previous Explanations for Loss of Plasticity

Not only are there several factors that could possibly contribute to loss of plasticity, there are also several explanations for this phenomenon. We survey the recent explanations of loss of plasticity below. In the next section, we present results showing that none of these explanations are sufficient to explain loss of plasticity across all problem settings we consider.

Decreasing update/gradient norm The simplest explanation for loss of plasticity is that the update norm goes to zero. This would mean that the parameters of the neural network stop changing, eliminating all plasticity. This tends to occur with a decrease in the norm of the features for particular layers (Abbas et al., 2023; Nikishin et al., 2022).

Dormant Neurons Another explanation for loss of plasticity is a steady decrease in the proportion of active neurons, namely, the dormant neuron phenomenon (Sokar et al., 2023). It is hypothesized that a decrease in the number of active neurons also decreases a neural network's expressivity, potentially leading to loss of plasticity.

Decreasing representation rank Related to the effective capacity of a neural network, lower representation rank suggests that fewer features are being represented by the neural network (Kumar et al., 2021). It has been observed that decreasing representation rank is sometimes correlated with loss of plasticity (Lyle et al., 2023; Kumar et al., 2023; Dohare et al., 2023a).

Increasing parameter norm An increasing parameter norm is sometimes associated with loss of plasticity in both continual supervised and continual reinforcement learning (Nikishin et al., 2022; Dohare et al., 2023a), but it is not necessarily a cause (Lyle et al., 2023). It is not clear why the parameter norms increase and lead to loss of plasticity, perhaps suggesting a slow divergence in the training dynamics.

3. Counterexamples for Previous Explanations

In this section, we investigate the explanations for loss of plasticity described in Section 2 and we provide counterexamples for them, showing that they fail to fully explain loss of plasticity. To do so, we use a linearly separable subset of the MNIST dataset (LeCun et al., 2010), in which the labels of each image are periodically shuffled. While MNIST is a simple classification problem, label shuffling highlights the difficulties associated with preserving plasticity (see Lyle et al., 2023; Kumar et al., 2023). We focus on this problem for its simplicity, showing that even in a setting where linear function approximation is sufficient, one can find counterexamples to the previous explanations in the literature for loss of plasticity. We emphasize that the goal here is merely to uncover simple counterexamples that refute proposed explanations for loss of plasticity, not to investigate the phenomenon more broadly. In Section 6, we extend our investigation of loss of plasticity to larger scale benchmarks.

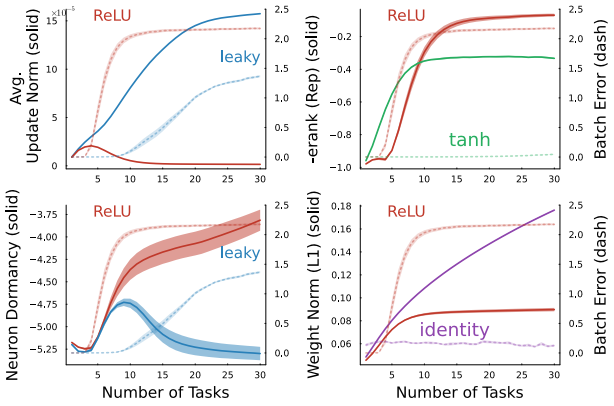


Figure 1. **Inconsistencies of previous explanations for loss of plasticity on Random Label MNIST (subset)**. The explanations on the left are not consistent because both `ReLU` and `leaky-ReLU` suffer from loss of plasticity. On the right, there is no loss of plasticity for `tanh` and `identity` but the corresponding explanations predict that they do. All results have a shaded region corresponding to a 95% confidence interval of the mean over 30 runs.

Methods In this experiment, we vary only the activation function between `ReLU`, `leaky-ReLU`, `tanh` and the `identity`. As noted in Section 2.1, previous work has found that the activation function has a significant effect on the plasticity of the neural network. We measure the error across all observations at the end of each task. Each task lasts 200 epochs, which is sufficient for neural networks with any of the considered activation functions to achieve low error on the first few tasks using a random initialization.

Results The main result of this experiment can be found in Figure 1. Our findings show that none of the aforementioned explanations of loss of plasticity explain the phenomenon. All non-linear activation functions can

achieve low error on the first few tasks, but for `ReLU` and `leaky-ReLU`, the error increases and eventually becomes worse than the neural network with identity activation (which is incapable of feature learning).[2] Despite some non-linear activation functions losing plasticity, the explanations on the left side of Figure 1 fail to predict loss of plasticity consistently. A decreasing update norm, for example, may seem like an intuitive explanation of loss of plasticity. However, in the top-left plot, we see that the update norm consistently increases for the `leaky-ReLU` activation function, making the explanation inconsistent. For the right side of Figure 1, the corresponding explanation predicts loss of plasticity for `tanh` and `identity` but we see it does not actually occur. The rank of the representation (plotted as a negative for uniformity with other explanations), another popular candidate explanation, decreases for the `tanh` activation despite no loss of plasticity in this problem.

Because feature rank is such a predominant explanation for loss of plasticity, we provide an additional counter-example showing that the feature rank is also not a sufficient explanation; rather, it is a symptom of a deeper problem. We re-run the previous experiment using a regularizer, $J_{\text{feature-reg}}(\Phi) = \sigma_1^2(\Phi) - \sigma_d^2(\Phi)$, that encourages the feature representation to be full rank (Kumar et al., 2021). The results, in Figure 2, show that regularization increases the feature rank, but that this is not sufficient to prevent loss of plasticity. For example, take the rank of the feature representation between tasks 5 and 10; although it increases in that period, the error increases, which means plasticity is still being lost.

Summary The previous explanations are not consistent because there exists at least one activation such that the trend in the training error does not agree with the trend in the explanation (see Appendix A for additional analysis). A maybe surprising finding is that the deep linear network (a neural network with an `identity` activation function) is able to maintain a low training error across all tasks for this problem. A deep linear network has more parameters than a linear function, but it can only represent linear functions. This is sufficient to solve each task because the number of data points (1280) is smaller than the effective dimensionality of the network ($d_{in} \times d_{out} = 7840$). The deep linear network's ability to preserve plasticity is surprising because the training dynamics of a deep linear network are non-linear and similar to a deep non-linear network (Saxe et al., 2014). The fact that loss of plasticity only occurs with non-linear activations suggests that the curvature introduced by the non-linearities is crucial in explaining loss of plasticity.

[2]While the neural network with `tanh` activations does not lose plasticity in this experiment, in Section 6 we show that it does lose plasticity when we consider the full MNIST dataset.

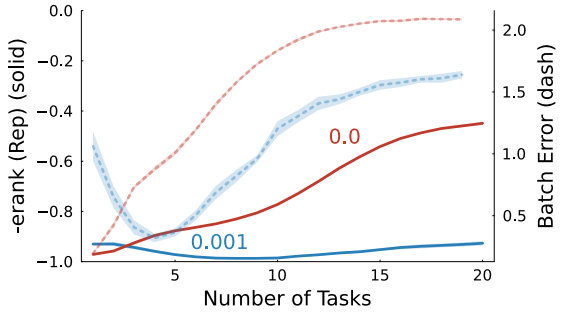


Figure 2. **Effect of feature rank regularization is maintaining plasticity.** Loss of plasticity still occurs with `leaky-ReLU` and feature rank regularization, despite the fact that the feature rank remains high. All results have a shaded region corresponding to a 95% confidence interval of the mean over 30 runs.

4. Measuring the Curvature of a Changing Optimization Landscape

A missing piece in the previously proposed explanations is the curvature of the optimization landscape. While previous work pointed out that curvature is connected to plasticity (Lyle et al., 2023), our work specifically posits that a reduction in the number of curvature directions coincides with loss of plasticity. In Section 6 we show that loss of plasticity occurs when, at the start of a new task, the optimization landscape has a diminishing number of curvature directions.

The optimization landscape in continual learning is not easy to characterize because it can change without the parameters changing. Unlike supervised learning, where the data distribution is stationary, the data distribution underlying the observations and targets will change in the continual learning setting. Thus there can be changes in the objective, gradient and Hessian that is due to the data changing and not due to parameter changes.

Before presenting empirical evidence of the relationship between plasticity and curvature, we note that there are several notions of curvature in the literature. The local curvature of the optimization landscape at a particular parameter θ is expressed by the Hessian of the objective function, $H_t(\theta) = \nabla^2_\theta J_t(\theta)\big|_{\theta=\theta_t}$.[3] Different measures of curvature correspond to different functions of this Hessian matrix. One common measure of curvature is the sharpness, given by the maximum eigenvalue of the Hessian (Keskar et al., 2016; Cohen et al., 2021). Sharpness is coarse-grained, it only gives the magnitude of the vector of maximal curvature and it fails to characterize other directions. Another measure, and the one that this paper investigates, is the effective rank of the Hessian matrix, which counts the effective number of directions of curvature.

[3]We omit the dependence on data in the training objective and the Hessian, instead indexing both by time.

We are interested in how the curvature of the optimization landscape changes when the task changes. Of particular interest is the rank of the Hessian after a task change. If it is decreasing, then there are fewer directions of curvature to explore the parameter space and to learn on the new task. For simplicity, and in alignment with our experiments, we assume that each task has an update budget of U iterations. Thus, the training objective on the K-th task is stationary for U steps. When the task changes, at $t = UK+1$, the Hessian changes due to changes in the data—and not due to changes in the parameters. We measure the rank at the beginning of the task by the *effective rank*, $\texttt{erank}\left(H_{UK+1}(\theta)\right)$, where $\texttt{erank}(M) = \min\left\{j : \frac{\sum_{i=1}^{j}\sigma_i(M)}{\sum_{i=1}^{d}\sigma_i(M)} > 0.99\right\}$ is the effective rank and $\{\sigma_i(M)\}_{i=1}^{d}$ are the singular values arranged in decreasing order. The effective rank specifies the number of basis vectors needed to represent 99% of image of the matrix M (Yang et al., 2019; Kumar et al., 2021).

4.1. Approximating the Hessian Rank

Neural networks typically have a large number of parameters, requiring approximations to the Hessian due to the massive computational overhead for producing the matrix. Diagonal approximations are employed to capture curvature information relevant for optimization (Elsayed & Mahmood, 2022; LeCun et al., 1989), but are full rank unless the parameter gradients become zero, which typically does not occur in classification. There are low-rank approximations of the Hessian (Le Roux et al., 2007), these too are problematic for our analysis because we aim to measure the rank of the Hessian and cannot presuppose that it is low-rank. Lastly, stochastic Lanzcos methods are able to efficiently approximate the smallest and largest eigenvalues (Ghorbani et al., 2019), but they cannot efficiently estimate the middle bulk of eigenvalues which can determine the rank.

To approximate the Hessian rank, we use the an outer-product approximation of m per-sample gradients, $\mathbf{H} \approx \hat{\mathbf{H}} = \sum_i^m g_i g_i^\intercal$, where $g_i = \nabla_\theta J(\theta, x_i, y_i)$ is the gradient with respect to a single datapoint (x_i, y_i). This approximation is useful for estimating the rank because if v is in the nullspace of the Hessian, $\hat{\mathbf{H}}v = 0$, then it is a direction of zero curvature and orthogonal to the per-sample gradients, $g_i^\intercal v = 0$. Thus, the vector is in the nullspace of the outer-product approximation and $\hat{\mathbf{H}}v = 0$. Of course, $\texttt{rank}(\hat{\mathbf{H}}) \leq M$ and $M << d$ means that the approximation will underestimate the rank. Our interest is in the relative decrease in the rank. We will report the effective rank divided by the maximum rank because the exact number of curvature directions is not relevant for our results.

The outer-product approximation also avoids the computational demands of the singular value decomposition needed to compute the effective rank. First, we rewrite the approximation $\hat{\mathbf{H}} = \sum_{i=1}^{M} g_i g_i^\intercal = \mathbf{G}\mathbf{G}^\intercal$, where $\mathbf{G} = [g_1, \ldots, g_m] \in \mathbb{R}^{d\times M}$ is the matrix of per-sample gradients. Then, because $\hat{\mathbf{H}}$ is a Gram matrix, we have that $\texttt{rank}(\mathbf{G}\mathbf{G}^\intercal) = \texttt{rank}(\mathbf{G}^\intercal\mathbf{G})$. This is useful because $\mathbf{G}^\intercal\mathbf{G} \in \mathbb{R}^{M\times M}$ and M is much smaller than d.

Another name for this approximation is the empirical Fisher information matrix, and it has been argued that it should not be used as a replacement for the Hessian as a pre-conditioner in second-order optimization because it is not guaranteed to capture the curvature information of the Hessian (Kunstner et al., 2019). Recent work studying neural network generalization, however, argues that the inner product of the per-example gradients can be useful in understanding neural network generalization and learning dynamics (Fort et al., 2019; Lyle et al., 2022). The matrix of gradient inner products, equivalently $\mathbf{G}^\intercal\mathbf{G}$, was also used to assess gradient covariance in continual learning (Lyle et al., 2023). Thus, the relative rank of the gradient outer-products provides a reasonable approximation to the relative rank of the Hessian, which we demonstrate empirically in the next section.

4.2. Validating the Hessian Rank Approximation

We evaluate the approximation to the Hessian rank in a simple problem where we can efficiently calculate the full Hessian and its rank. The problem is similar to the experiments in Section 3, except we also apply a stochastic projection matrix to the MNIST images to reduce the input dimension and overall parameter count.

We compare the approximation quality of the Hessian rank using three different methods: 1) Empirical Fisher (our approach), 2) Fisher, and, 3) Gauss-Newton. We measure the rank of the exact Hessian and the rank of the Hessian approximation at the beginning of each new task. Next, we normalize each rank by its corresponding maximum possible rank. To measure the approximation quality, we plot the absolute difference between the relative effective Hessian ranks. Our results in Figure 3 show that the proposed empirical Fisher approximation to the Hessian rank is particularly accurate in estimating the rank in the first few tasks, which is when loss of plasticity occurs. As plasticity degrades in later tasks, the approximation quality worsens but still accurately represents the overall trend of the true Hessian rank.

Comparisons for other neural networks, further details, and figures demonstrating the dynamics of the Hessian approximation can be found in Appendix C.2. We use this Hessian rank approximation to explain loss of plasticity in continual supervised learning in the rest of our experiments.

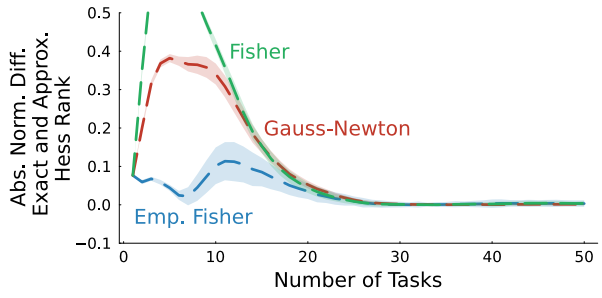


Figure 3. **Comparison between different methods for approximating the Hessian rank.** The empirical Fisher approximation to the Hessian rank is highly accurate in the first few tasks, which is when loss of plasticity occurs. When plasticity worsens in later tasks, the approximation quality marginally worsens. Overall, the empirical Fisher is an accurate and efficient approximation to the Hessian rank.

5. Preserving Curvature with Regularization

In the previous section, we claimed that loss of curvature may explain loss of plasticity. Regularization is commonly used to improving the conditioning of matrices (Benning & Burger, 2018). This does not immediately imply that regularization preserves plasticity because we are interested in minimizing the unregularized objective, and preserving the rank of the Hessian with respect to the unregularized objective. Our central claim is that regularization also preserves the rank of the unregularized Hessian, and allows neural networks to preserve plasticity.[4]

If curvature is lost over the course of learning, then one solution to this problem is to regularize towards the curvature present at initialization. While explicit Hessian regularization would be computationally costly, previous work has found that even weight decay can mitigate loss of plasticity (Dohare et al., 2021; Lyle et al., 2021; Kumar et al., 2023), without attributing this benefit to preserving directions of curvature. These methods, however, do more than just prevent loss of curvature, they also prevent parameters from growing large (subject to the regularization parameter's strength). Weight decay, for example, mitigate loss of plasticity but also prevent the parameters from deviating far from the origin. The restriction that weight decay imposes on the update requires careful tuning of the regularization strength as we show in Section 6 and Appendix C.6.

We propose a new regularizer that is simple and that gives the parameters more leeway for moving from the initialization, while preserving the desirable plasticity and curvature properties of the initialization. Our regularizer penalizes the distribution of parameters if it is far from the distribution of the randomly initialized parameters. At initialization, the parameters at layer l are sampled i.i.d. $\theta_{i,j} \sim p^{(l,0)}(\theta)$ according to some pre-determined distribution, such as the Glorot initialization (Glorot & Bengio, 2010). The distribution of parameters at iteration t during training and for any particular layer, denoted by $p^{(l,t)}$, is no longer known (the parameters may not be independent nor identically distributed). However, it is still possible to regularize the empirical distribution towards the initialization distribution by using the empirical Wasserstein metric (Bobkov & Ledoux, 2019). We denote the flattened parameter matrix for layer l at time t by $\bar{\theta}^{(l,t)}$. The squared Wasserstein-2 distance between the distribution of parameters at initialization and the current parameter distribution is defined as,

$$\mathcal{W}_2^2\left(p^{(l,0)}, p^{(l,t)}\right) = \sum_{i=1}^{d}\left(\bar{\theta}_{(i)}^{(l,t)} - \bar{\theta}_{(i)}^{(l,0)}\right)^2.$$

The order statistics of the parameter is denoted by $\theta_{(i)}^{(l,t)}$ and represents the i-th smallest parameter at time t for layer l. In the above equation, we are taking the L2 difference between the order statistics of each layer's parameters at initialization and at iteration t during training. The *Wasserstein regularizer* uses the empirical Wasserstein distance for each layer of the neural network.

A recent alternative, regenerative regularization, regularizes the neural network parameters towards their initialization (Kumar et al., 2023). The regenerative regularizer mitigates loss of plasticity, but it also prevents the neural network parameters from deviating far from the initialization. Unlike the regenerative regularizer, the Wasserstein regularizer takes the difference of the order statistics. Thus, the regenerative regularizer is always larger because the Wasserstein regularizer takes the difference in the sorted values, $\sum_{i=1}^{d}\left(\bar{\theta}_{(i)}^{(l,t)} - \bar{\theta}_{(i)}^{(l,0)}\right)^2 < \sum_{i=1}^{d}\left(\bar{\theta}_{i}^{(l,t)} - \bar{\theta}_{i}^{(l,0)}\right)^2$. As we show in Appendix C.5, the Wasserstein regularizer allows the network parameters to deviate further from the initialization. This means that learning with the Wasserstein regularizer requires fewer iterations while achieving a lower error compare to other regularizers (see inter-task learning curves, Appendix C.8).

6. Experiments: Effect of Curvature and Regularization in Plasticity Benchmarks

We now validate our claim that loss of curvature, as measured by the reduction in the rank of the Hessian, explains loss of plasticity. Our experiments use the four most common continual learning benchmarks in which loss of plasticity has been reported (see Appendix B for further details):

- Permuted MNIST: A commonly used benchmark across continual learning where the pixels are peri-

[4] All measurements of the Hessian rank are with respect to the unregularized objective.

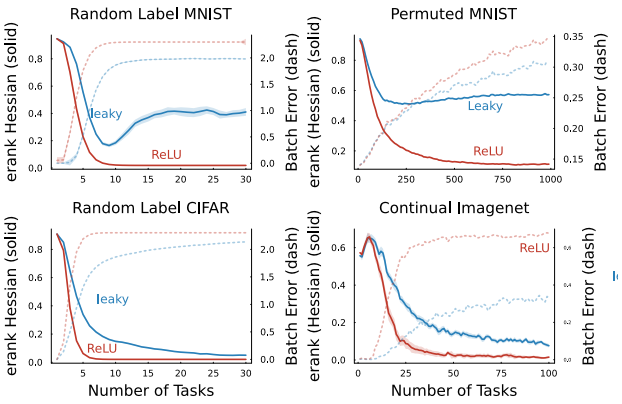


Figure 4. **Validating that a reduction in the directions of curvature is a consistent explanation for loss of plasticity.** A reduction in the directions of curvature co-occurs with loss of plasticity. `leaky-ReLU` preserves plasticity for longer but is unable to recover its directions of curvature.

odically permuted (Goodfellow et al., 2013; Zenke et al., 2017; Kumar et al., 2023; Dohare et al., 2023a; Elsayed & Mahmood, 2023).

- Random Label MNIST: A more difficult task change where all labels are randomized (Kumar et al., 2023; Lyle et al., 2023; Elsayed & Mahmood, 2023). This problem was used in Section 3, but in this section we use the entire MNIST dataset.

- Random Label CIFAR-10 (Krizhevsky, 2009): An increasingly common problem setting for studying the plasticity of convolutional neural networks due to the relative complexity of images in CIFAR (Kumar et al., 2023; Lyle et al., 2023; Sokar et al., 2023).

- Continual ImageNet (Dohare et al., 2023a): A sequence of 500 binary classification tasks from the ImageNet dataset (Russakovsky et al., 2015) where none of the classes are shared between tasks.

To provide evidence for the claim that curvature explains loss of plasticity, we conduct an in-depth analysis of the change of curvature in continual supervised learning. We first show that curvature is a consistent explanation across different problem settings. Afterwards, we investigate the role of curvature on learning to find that the gradient tends to overlap with the shrinking top-subspace of the Hessian (to a degree depending on the activation function). Lastly, we show that regularization, which has been demonstrated to be effective in mitigating loss of plasticity, also mitigates loss of curvature.

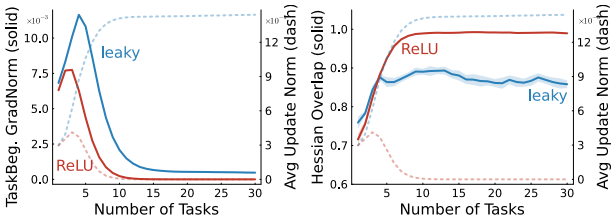


Figure 5. **Curvature explains why the average update norm increases when using `leaky-ReLU` despite loss of plasticity.** Left: `leaky-ReLU` has an increasing average update norm despite a decrease in the gradient norm at the beginning of a task. Right: gradients with `leaky-ReLU` have less overlap with the low-rank Hessian, meaning that updates occur in more directions than with `ReLU`.

6.1. Does Loss of Curvature Explain Loss of Plasticity?

We present the results on the four problem settings in Figure 4. This is the same setting as the results in Section 3, but with the full MNIST dataset (see Appendix C.3 for results on all activation functions). Loss of curvature tends to co-occur with loss of plasticity for the non-linear activations, providing a consistent explanation of the phenomenon compared to previous explanations.

6.2. How Does Loss of Curvature Affect Learning?

Having demonstrated that loss of curvature co-occurs with loss of plasticity, we now investigate how loss of curvature affects the gradients and learning. Our goal is to explain why the update norms can be increasing for `leaky-ReLU` despite loss of plasticity. In Figure 5 (Left), we see that the gradient norm at the beginning of each task is decreasing, which neither explains loss of plasticity nor the increasing update norm. In the right plot, we measure the overlap between the gradient and the (top subspace) Hessian-gradient product at the beginning of a task given by $\frac{g^T H g}{\|g\| \|Hg\|}$.[5] This measures whether the gradient is contained in the top subspace of the Hessian (Gur-Ari et al., 2018). For `leaky-ReLU`, the gradient has less overlap with the top subspace of its Hessian. This means that updates with `leaky-ReLU` explore a higher dimensional space than than either `tanh` or `ReLU`, explaining why its average update norm is higher.

6.3. Can Regularization Preserve Curvature?

We now investigate whether regularization prevents loss of plasticity and, if it does, whether it also preserves directions of curvature. Our results for the four problem settings are summarized in Figures 6. We see that the Wasserstein is able to preserve plasticity, achieving similar error to the regenerative regularizer on the easier MNIST problems and

[5] We zero out singular values smaller than the effective rank to ensure that the gradient overlaps with the top-subspace Hessian.

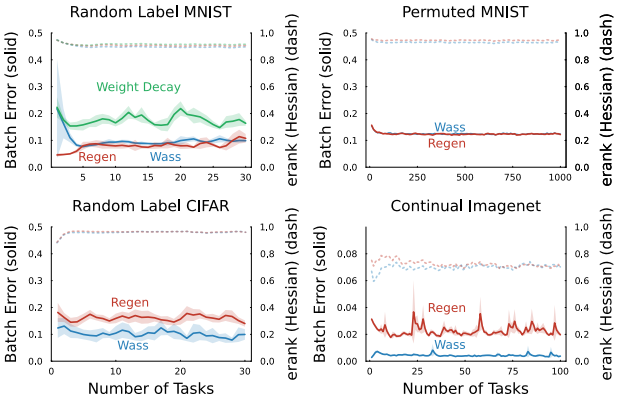


Figure 6. **Regularization preserves plasticity and directions of curvature.** Wasserstein and regenerative regularizers are effective at preserving plasticity and curvature. On harder problems (bottom), the Wasserstein regularizer achieves a lower error.

achieving the lowest error on Random Label CIFAR and Continual ImageNet. The success of the Wasserstein regularizer can be seen from two perspectives: 1) parameters can move further from initialization (see Appendix C.5) and 2) reduced sensitivity to the regularization strength (see Appendix C.6). The inter-task learning curves reveal that learning with the Wasserstein regularizer not only achieves a lower error, but that learning can require fewer iterations (see Appendix C.8). Lastly, we find that the feature rank is often decreasing for the regularized neural networks, which further demonstrates its inconsistency as an explanation for loss of plasticity (see Appendix C.4).

6.4. Does Scale Help Preserve Plasticity & Curvature?

To investigate the role of neural network scale, we ablate different neural network widths and depths. The results in Figure 7 show that increasing both the depth and width of the neural network only delays loss of plasticity. In Figure 8, we test whether loss of plasticity occurs in CIFAR-10 using a much larger network with batch normalization, ResNet18 (He et al., 2016). Unlike the previously considered convolutional networks, the ResNet is able to decrease the error on the first few tasks despite training for only 20 epochs. However, loss of plasticity still occurs without regularization. With regularization, the ResNet is able to achieve an error level slightly higher than the best error that the unregularized version can achieve.

7. Discussion

We have demonstrated how loss of curvature directions is a consistent explanation for loss of plasticity when compared to previous explanations offered in the literature. One limitation of our work is that we study an approximation to the Hessian. Our experiments suggest that this approximation

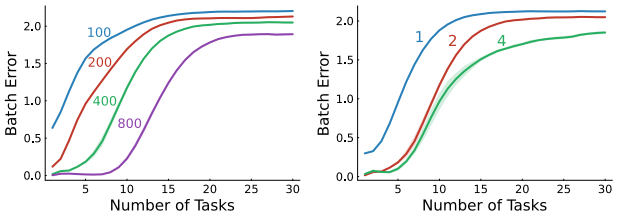


Figure 7. **Effect of width and depth on loss of plasticity.** Increasing either the width of the hidden layer or the depth (number of hidden layers) in a neural network delays loss of plasticity, and marginally lowers the error plateau, but does not eliminate loss of plasticity. Right: Varying the width, while keeping the depth fixed at 4. Left: Varying the depth, while keeping the width fixed at 800.

of the Hessian is enough to capture changes in the number of curvature directions, but more insight may be found from theoretical study of the entire Hessian. Another limitation is that it is not clear what drives neural networks to lose curvature directions during training. Understanding the dynamics of training neural networks with gradient descent, however, is an active research area even in supervised learning. It will be increasingly pertinent to understand what drives neural network training dynamics to lose curvature directions so as to develop principled algorithms for continual learning.

Our experimental evidence demonstrates that, when loss of plasticity occurs, there is a reduction in curvature as measured by the rank of the Hessian at the beginning of subsequent tasks. When loss of plasticity does not occur, curvature remains relatively constant. Unlike previous explanations, this phenomenon is consistent across different datasets, non-stationarities, step-sizes, and activation functions. Lastly, we investigated the effect of regularization on plasticity, finding that regularization tends to preserve curvature but can be sensitive to the regularization strength. We proposed a simple distributional regularizer that proves effective in maintaining plasticity across the problem settings we consider, while maintaining curvature and being less hyperparameter sensitive.

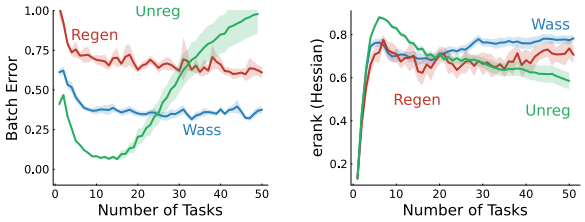


Figure 8. **ResNet18 without regularization still suffers from loss of plasticity.** Despite a much higher higher parameter count and batch normalization, the ResNet is not able to maintain its initial error without regularization due to a reduction in the number of curvature directions, as measured by the rank of the Hessian.

Acknowledgments

We thank Shibhansh Dohare, Khurram Javed, Farzane Aminmansour and Mohamed Elsayed for early discussions about loss of plasticity. The research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC), the Canada CIFAR AI Chair Program, the Digital Research Alliance of Canada and Alberta Innovates Graduate Student Scholarship.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Abbas, Z., Zhao, R., Modayil, J., White, A., and Machado, M. C. Loss of plasticity in continual deep reinforcement learning. In *Conference on Lifelong Learning Agents*, 2023.

Ash, J. T. and Adams, R. P. On warm-starting neural network training. In *Advances in Neural Information Processing Systems*, 2020.

Benning, M. and Burger, M. Modern regularization methods for inverse problems. *Acta numerica*, 27:1–111, 2018.

Berariu, T., Czarnecki, W., De, S., Bornschein, J., Smith, S., Pascanu, R., and Clopath, C. A study on the plasticity of neural networks. *CoRR*, abs/2106.00042, 2021.

Bobkov, S. and Ledoux, M. *One-dimensional empirical measures, order statistics, and Kantorovich transport distances*, volume 261. American Mathematical Society, 2019.

Cohen, J., Kaur, S., Li, Y., Kolter, J. Z., and Talwalkar, A. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations*, 2021.

Dohare, S., Sutton, R. S., and Mahmood, A. R. Continual backprop: Stochastic gradient descent with persistent randomness. *CoRR*, abs/2108.06325v3, 2021.

Dohare, S., Hernandez-Garcia, J. F., Rahman, P., Sutton, R. S., and Mahmood, A. R. Maintaining plasticity in deep continual learning. *CoRR*, abs/2306.13812, 2023a.

Dohare, S., Lan, Q., and Mahmood, A. R. Overcoming policy collapse in deep reinforcement learning. In *Sixteenth European Workshop on Reinforcement Learning*, 2023b.

Elsayed, M. and Mahmood, A. R. Hesscale: Scalable computation of hessian diagonals. *CoRR*, abs/2210.11639v2, 2022.

Elsayed, M. and Mahmood, A. R. Utility-based perturbed gradient descent: An optimizer for continual learning. *CoRR*, abs/2302.03281v2, 2023.

Fort, S. and Ganguli, S. Emergent properties of the local geometry of neural loss landscapes. *CoRR*, abs/1910.05929, 2019.

Fort, S., Nowak, P. K., Jastrzebski, S., and Narayanan, S. Stiffness: A new perspective on generalization in neural networks. *CoRR*, abs/1901.09491v3, 2019.

Fukushima, K. Cognitron: A self-organizing multilayered neural network. *Biological cybernetics*, 20(3-4):121–136, 1975.

Ghorbani, B., Krishnan, S., and Xiao, Y. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning*, 2019.

Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics*, 2010.

Goodfellow, I. J., Mirza, M., Xiao, D., Courville, A., and Bengio, Y. An empirical investigation of catastrophic forgetting in gradient-based neural networks. *CoRR*, abs/1312.6211, 2013.

Gur-Ari, G., Roberts, D. A., and Dyer, E. Gradient descent happens in a tiny subspace. *CoRR*, abs/1812.04754v1, 2018.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Conference on Computer Vision and Pattern Recognition*, 2016.

Hochreiter, S. and Schmidhuber, J. Flat minima. *Neural computation*, 9(1):1–42, 1997.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. An empirical analysis of compute-optimal large language model training. *Advances in Neural Information Processing Systems*, 2022.

Igl, M., Farquhar, G., Luketina, J., Boehmer, W., and Whiteson, S. Transient non-stationarity and generalisation in deep reinforcement learning. In *International Conference on Learning Representations*, 2021.

Javed, K. and White, M. Meta-learning representations for continual learning. *Advances in Neural Information Processing Systems*, 2019.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. On large-batch training for deep learning: Generalization gap and sharp minima. *CoRR*, abs/1609.04836, 2016.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Kumar, A., Agarwal, R., Ghosh, D., and Levine, S. Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In *International Conference on Learning Representations*, 2021.

Kumar, S., Marklund, H., and Roy, B. V. Maintaining plasticity via regenerative regularization. *CoRR*, abs/2308.11958v1, 2023.

Kunstner, F., Hennig, P., and Balles, L. Limitations of the empirical fisher approximation for natural gradient descent. In *Advances in Neural Information Processing Systems*, 2019.

Le Roux, N., Manzagol, P.-A., and Bengio, Y. Topmoumoute online natural gradient algorithm. *Advances in Neural Information Processing Systems*, 2007.

LeCun, Y., Denker, J., and Solla, S. Optimal brain damage. *Advances in Neural Information Processing Systems*, 1989.

LeCun, Y., Cortes, C., and Burges, C. MNIST handwritten digit database. *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, 2010.

Lyle, C., Rowland, M., and Dabney, W. Understanding and preventing capacity loss in reinforcement learning. In *International Conference on Learning Representations*, 2021.

Lyle, C., Rowland, M., Dabney, W., Kwiatkowska, M., and Gal, Y. Learning dynamics and generalization in deep reinforcement learning. In *International Conference on Machine Learning*, 2022.

Lyle, C., Zheng, Z., Nikishin, E., Avila Pires, B., Pascanu, R., and Dabney, W. Understanding plasticity in neural networks. In *International Conference on Machine Learning*, 2023.

Nair, V. and Hinton, G. E. Rectified linear units improve restricted Boltzmann machines. In *International Conference on Machine Learning*, 2010.

Nikishin, E., Schwarzer, M., D'Oro, P., Bacon, P.-L., and Courville, A. The primacy bias in deep reinforcement learning. *CoRR*, abs/2205.07802v1, 2022.

Ring, M. B. *Continual learning in reinforcement environments*. The University of Texas at Austin, 1994.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 2015.

Saxe, A., McClelland, J., and Ganguli, S. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Represenatations*, 2014.

Shang, W., Sohn, K., Almeida, D., and Lee, H. Understanding and improving convolutional neural networks via concatenated rectified linear units. In *international Conference on Machine Learning*, 2016.

Sokar, G., Agarwal, R., Castro, P. S., and Evci, U. The dormant neuron phenomenon in deep reinforcement learning. In *International Conference on Machine Learning*, 2023.

Thrun, S. Lifelong learning algorithms. In *Learning to Learn*, pp. 181–209. Springer, 1998.

Xu, B., Wang, N., Chen, T., and Li, M. Empirical evaluation of rectified activations in convolutional network. *CoRR*, abs/1505.00853, 2015.

Yang, Y., Zhang, G., Xu, Z., and Katabi, D. Harnessing structures for value-based planning and reinforcement learning. In *International Conference on Learning Representations*, 2019.

Zenke, F., Poole, B., and Ganguli, S. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, 2017.

Zilly, J., Achille, A., Censi, A., and Frazzoli, E. On plasticity, invariance, and mutually frozen weights in sequential task learning. *Advances in Neural Information Processing Systems*, 2021.

Ziyin, L. Symmetry leads to structured constraint of learning. *CoRR*, abs/2309.16932v1, 2023.

Appendix

A. Additional analysis on counter-examples

In the body of paper, we provided a high-level analysis of Figure 1, and concluded that none of the previous explanations for loss of plasticity (i.e. increasing error) is consistent amongst the different activation fucntions. Here, we aim to compliment that high-level analysis by providing detailed explanation on how each metric is inconsistent with the batch error.

1. **Average Update Norm** (top-left): The plot measures the average L1 norm of the parameter updates, and it is predicted that a decrease in the update norm leads to loss of plasticity. Both `Leaky-ReLU` and `ReLU` exhibit the opposite trend in their update norm: the former is increasing its average update norm and the latter is decreasing. But, both activation functions have an increasing error and thus suffer from loss of plasticity. Hence, the update norm is an inconsistent explanation for loss of plasticity

2. **Effective Rank of Representation** (top-right): The plot measures the normalized effective rank of the representation (the last hidden layer that is mapped linearly to the output space), and it is predicted that a decrease in the feature rank leads to loss of plasticity. For `ReLU`, the representation rank decreases along with the error increasing, which is what the effective rank explanation of plasticity predicts. The representation rank is inconsistent because `tanh` has an initial drop of its representation rank despite the error remaining constant. Hence, the representation rank is an inconsistent explanation for loss of plasticity.

3. **Dormant Neurons** (bottom-left): The plot measures neuron dormancy by the negative of the entropy of the normalized absolute value of the features for each task, which captures the notion of dormancy that activations can concentrate on a small subset of features. It is predicted that an increase in neuron dormancy will lead to loss of plasticity The plot shows that the `ReLU` activation has an increase in neuron dormancy and an increasing error, which is what neuron dormancy predicts. But, `leaky-ReLU` experiences plasticity loss and the neuron dormancy is non-decreasing. Hence, the dormant neuron phenomenon is an inconsistent explanation for loss of plasticity.

4. **Weight Norm** (bottom-right): The plot presents the L1 norm of the weights at the end of each task, and it is predicted that an increasing norm leads to loss of plasticity Both `ReLU` and `identity` provide counterexamples. For `ReLU`, the weight norm plateaus but loss of plasticity occurs. For `identity`, the weight norm increases seemingly indefinitely and yet, loss of plasticity does not occur. Hence, the weight norm is an inconsistent explanation for loss of plasticity.

B. Experimental Details

B.1. Random Label MNIST

Non-stationary variant of the ordinary (stationary) supervised classification problem on MNIST dataset. The source of non-stationarity in this problem is the periodical random shuffling of labels, irrespective of the original class labels. The dataset consists of 51200 uniformly sampled MNIST image-label pairs. We iterate over the dataset for 200 epochs in the experiments in the main paper, but ablate for different number of epochs in Section C.9. After 200 epochs, the labels will be reshuffled within the same dataset, producing the new task. Each gradient updates are performed with the batch of 256 datapoints, hence the update number of updates per epoch is 200 and the number of updates in the task is 40000. The architecture is a 3 hidden layer feed-forward neural network with widths $(256, 256, 256)$. We use the Adam optimizer with default hyperparameters. We average over 30 seeds for the unregularized experiments and average over 30 seeds for the regularized experiments. For the regularized experiments, we sweep over the regularization strength of $\{0.005, 0.001, 0.0005\}$. We use `leaky-ReLU` for all regularized experiments (except with the ResNet) due to its increased effectiveness in the continual learning setting.

B.2. Permuted MNIST

The overall problem framework is identical to the Random Label MNIST, except for the source of non-stationarity. The non-stationarity is introduced by reordering the positions of pixels in each input image, while label remains the same throughout the experiment. At the beginning of each task, the permutation of pixels are shuffled, and each input images are uniformly shuffled according to that permutation. For the regularized experiments, we sweep over the regularization strength of $\{0.01, 0.005, 0.001, 0.0005\}$. Other components of experiment do not vary from Random Label MNIST problem.

B.3. Random Label CIFAR-10

A non-stationary supervised classification problem using the CIFAR-10 dataset, similar to the Random Label MNIST problem. Similarly in the label-shuffled MNIST problem, this problem uniformly samples 38400 datapoints from CIFAR-10. The architecture uses 4 convolutional layers with stride 2 and $(16, 32, 64, 128)$ filters, before flattening and using a single layer feed-forward neural network with width (512). For the regularized ex-

periments, we sweep over the regularization strength of $\{0.01, 0.005, 0.001, 0.0005\}$. Other components of experiment do not vary from Random Label MNIST problem. The ResNet18 architecture (He et al., 2016) is unchanged, using `ReLU` and batch normalization. We train the network for a reduced number of epochs (20) to demonstrate that the ResNet can initially improve its plasticity before losing plasticity. The regularized ResNet uses a regularization strength of 0.005 which was the best regularization strength found on the smaller convolutional neural network.

B.4. Continual ImageNet

We use the Continual ImageNet environment introduced by (Dohare et al., 2023a). We train the same convolutional neural network as before, but for 250 epochs. For the regularized experiments, we sweep over the regularization strength of $\{0.01, 0.005, 0.001, 0.0005\}$. Other components of experiment do not vary from Random Label CIFAR problem.

C. Additional Results

C.1. Average Online Error Can Suggest Loss of Plasticity Even in Its Absence

Average online error is another metric for studying loss of plasticity, but it can misdiagnose the phenomenon. Even if a neural network maintains a consistent error at the end of a task, its online error can increase due to an increase in its error at the beginning of a task. But the error at the beginning of a task is not controllable, because it is due to a non-stationarity in the experience. Thus, we focus on the batch error at task end alone.

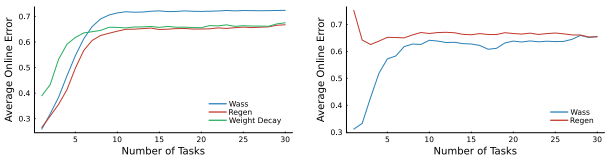


Figure 9. Regularization prevents loss of plasticity, in the sense that the error at the end of the task is constant. The average online error increases, because the error at the start increases.

C.2. Further Discussion and Results on Hessian Approximation

We use a stochastic projection matrix to reduce the dimensionality of the MNIST images to 36, then use a neural network with 3 hidden layers with 32 neurons. While the scale of this problem is small, its results with respect to plasticity remain strikingly similar to the larger scale problems in the main experiments.

The Fisher approximation differs from the empirical fisher approximation because it requires sampling from the predictive distribution induced by the classifier, and we use only 1 sample per datapoint. Sampling additional times would be more effective but less efficient. The Gauss-Newton approximation is $\mathbf{H} \approx J_f^T H_z J_f$, where J_f is the Jacobian of the neural network output and H_z is the Hessian of the loss function with respect to the prediction. we cannot interchange the inner and outerproduct because of the middle Hessian matrix. Thus, calculating the svd cannot be made efficient.

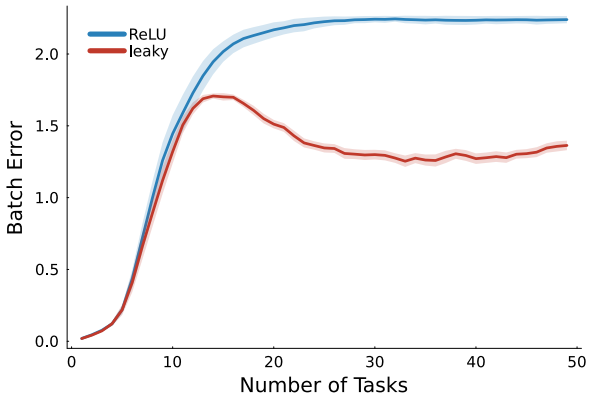


Figure 10. Neural networks suffer from loss of plasticity with both activations on low-dimensional projected MNIST.

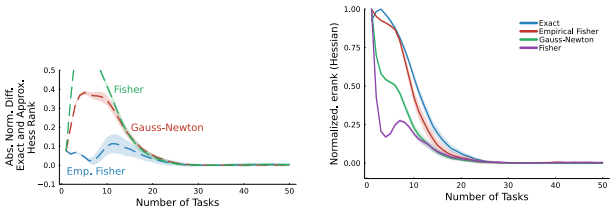


Figure 11. With `ReLU`, the empirical Fisher approximation accurately approximates the normalized rank throughout continual learning.

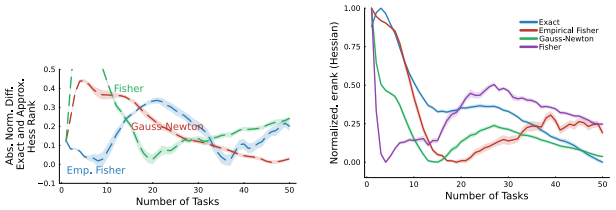


Figure 12. With `leaky-ReLU`, the empirical Fisher approximation accurately approximates the normalized rank when plasticity is being lost, which is sufficient as an indicator for loss of plasticity.

C.3. Results on All Activation Functions

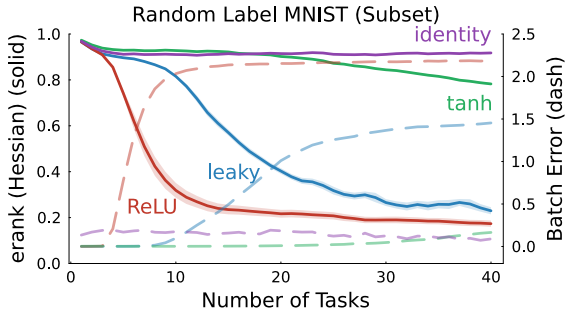


Figure 13. If we run the experiment in Section 3 for more tasks, `tanh` eventually loses plasticity and the Hessian rank accurately predicts this whereas the feature rank remains constant.

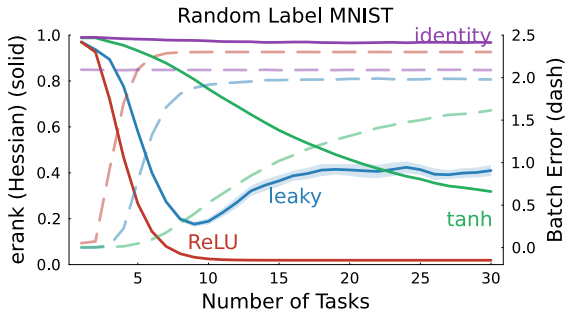


Figure 14. On the full dataset, the deep linear network does not have enough capacity to learn and it's error remains high but constant. All non-linear activation functions lose plasticity, which the Hessian rank correctly explains.

C.4. Parameter Regularization Preserves Plasticity But Does Not Always Control Feature Rank

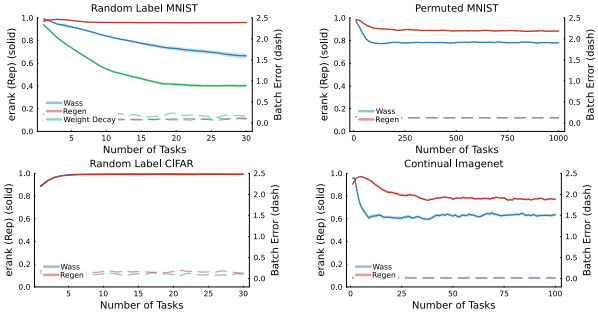


Figure 15. With regularization, the feature rank sometimes still decreases. The decrease is problem dependent, and only in the CIFAR problem does the feature rank increase.

C.5. Distances from Initialization With and Without Regularization

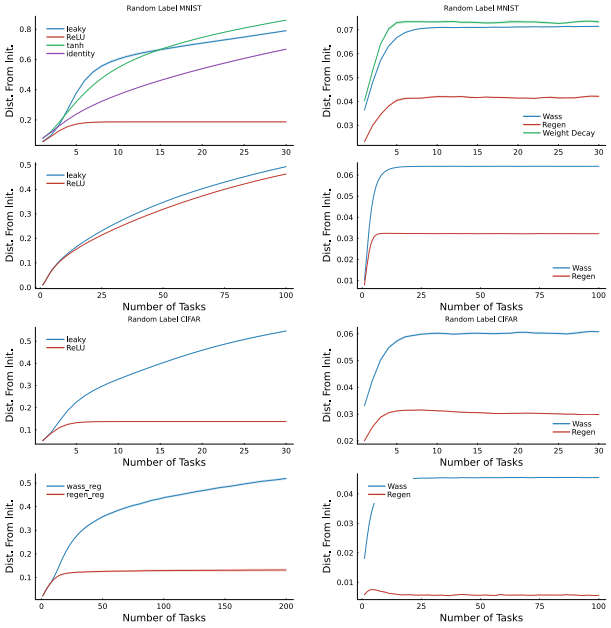


Figure 16. Regularization generally prevents neural network parameters from deviating from initialization compared to the unregularized setting. But, the Wasserstein regularizer minimizes a distributional distance that allows it to travel further from its initialization. Weight decay, although further from the initialization, is closer to the initialization distribution which can also lead to loss of plasticity (Zilly et al., 2021).

C.6. Regularizer Hyperparameter Sensitivity

The plots below show the batch error at the end of a task for different regularization strengths. Compared to weight decay and regenerative regularization, the Wasserstein regularizer is able to reach and maintain a lower error across most problems and activation functions.

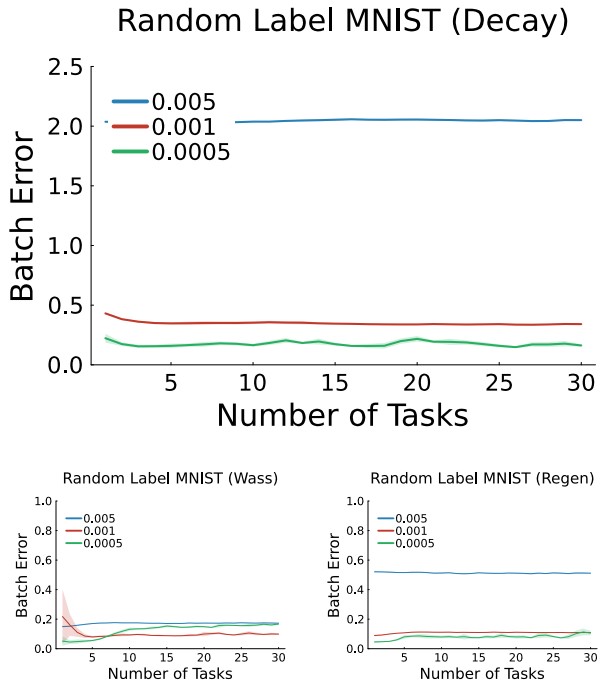


Figure 17. Learning curves on Random Label MNIST with different regularizers and different regularization strengths. The Wasserstein regularizer is less sensitive to the regularization strength

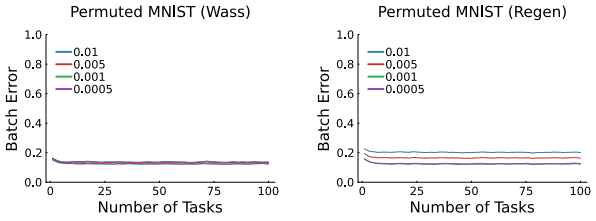


Figure 18. Learning curves on Permuted MNIST with different regularizers and different regularization strengths. The Wasserstein regularizer is less sensitive to the regularization strength

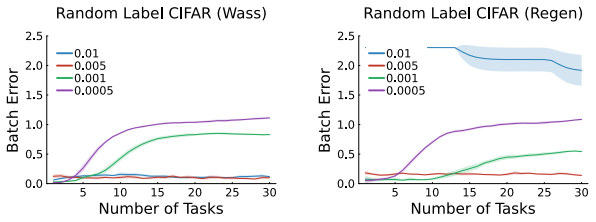


Figure 19. Learning curves on Random Label CIFAR with different regularizers and different regularization strengths. The Wasserstein regularizer is less sensitive to the regularization strength

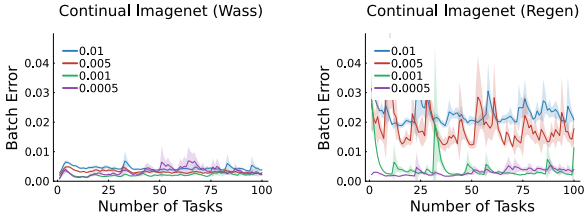


Figure 20. Learning curves on Continual Imagenet with different regularizers and different regularization strengths. The Wasserstein regularizer is less sensitive to the regularization strength

C.7. Inter-task Online Learning Curves Without Regularization

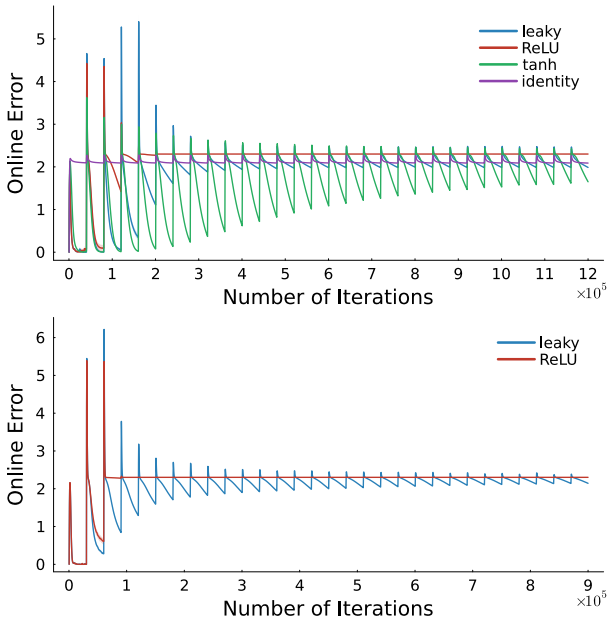


Figure 21. Intertask online learning curves on Random Label MNIST and Random Label CIFAR, without regularization.

C.8. Inter-task Online Learning Curves With Regularization

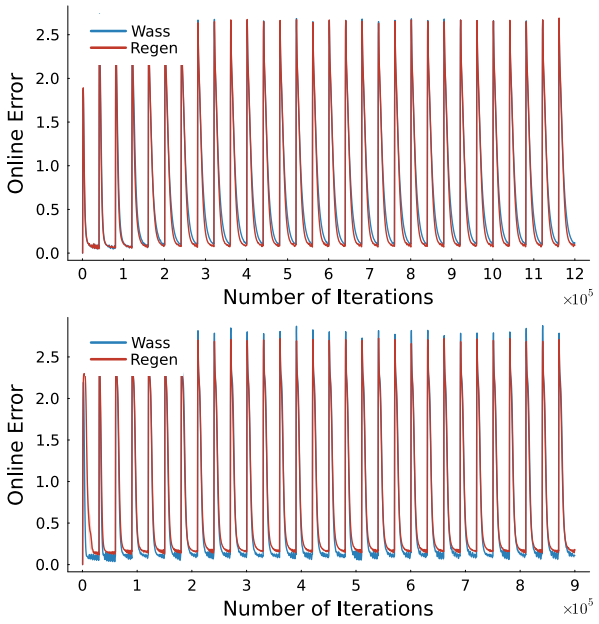


Figure 22. Intertask online learning curves on Random Label MNIST and Random Label CIFAR, with different regularizers.

C.9. Update Budget Effect on Plasticity

By varying the number of epochs in a task, the neural network is able to learn more on a task, perhaps allowing the neural network to escape from loss of plasticity. Unfortunately, the results in Figure 23 shows that increasing the number of epochs only marginally delays the onset of loss of plasticity. Plasticity loss still occurs, but reduction in curvature is a consistent predictor of this phenomenon.

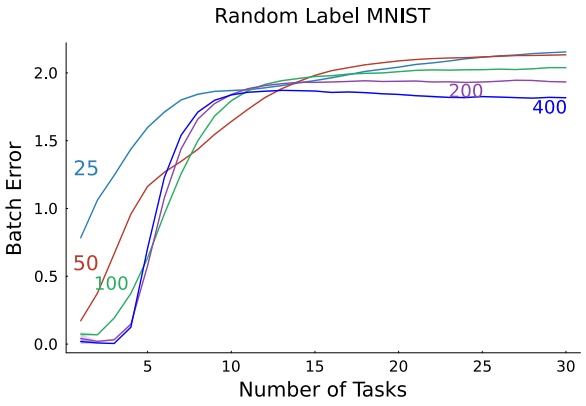


Figure 23. Ablating the number of epochs per task on Random Label MNIST. Loss of plasticity occurs when the number of epochs is small (25), despite not overfitting to the first few task. Loss of plasticity eventually also occurs when the number of epochs is large (400), but reduces the final error plateau.